

@pageflow/admin

Building admin UI in your plugins

AlfacodeTeam PhpServicePlatform — Sentinel

A practical guide to the admin foundation shipped by the Pageflow plugin: the persistent shell, the navigation registry, the settings-tab seam, and the list/table kit — and how any other plugin plugs into them without the foundation ever learning that plugin's name.

Every example is a complete, working file. The traps are real defects that have already been found in this codebase, not hypotheticals.

Contents

1	The idea in one page	2
2	Quick start: an admin page in one file	2
2.1	The page	2
2.2	The route	3
2.3	The controller	4
2.4	Wire it up	4
3	The layout	5
3.1	Attach it, do not wrap with it	5
3.2	AdminLayout options	5
3.2.1	providers: how a plugin injects its own dependency	6
3.3	AuthLayout	6
4	Shell data: the adminShell shared prop	7
4.1	Server side — share it once	7
4.2	Client side — read it	7
5	Contributing navigation	8
5.1	How visibility is decided	9
5.2	Badge resolution order	9
5.3	Nav-only, no page yet	9
6	Contributing a settings tab	10
7	Page header and footer	11
8	Lists, tables and mutations	12
8.1	Which one to reach for	12
8.2	A complete CRUD page	12
8.3	Server-side tables	14
8.4	Mutations: resourceRequest	14
9	Form controls and hooks	15
9.1	Controls	15
9.2	Hooks	16
10	Shipping both faces	16
10.1	Overriding a plugin page from a project	17
11	The trap list	18
12	Checklist and cheat sheet	18
12.1	Adding an admin page to a plugin	18
12.2	Cheat sheet	18
12.3	Further reading	19

1 The idea in one page

A plugin owns a business domain. It should ship the PAGES for that domain and nothing else — no sidebar, no header, no table component, no pagination, no empty-state illustration. All of that is @pageflow/admin, the admin foundation shipped inside the Pageflow plugin.

The hard part is the reverse direction. A sidebar has to know your plugin exists in order to show a link to it. A settings screen has to render your panel. If those relationships are written as imports, the foundation ends up importing every business domain in the application — which is exactly the coupling this architecture forbids on the PHP side.

So the direction is inverted. The foundation exposes REGISTRIES, your plugin writes into them at import time, and the surface discovers your file with a glob. Nothing in the foundation ever names your plugin.

What a plugin ships

```

plugins/hkm-plugin-rental/ui/|
  ui.json                  alias "@rental", surfaces map|
  index.ts                 barrel — shared bits other code may import|
  admin/|
    | nav.ts               ← registers your sidebar section (globbed)|
    |   Pages/Rental/*.tsx ← your admin pages (globbed)|
    | site/Pages/Rental/*.tsx ← your public pages (globbed)|
    |
    ▲ discovered, never imported by the foundation

```

You want to add...	Mechanism	Foundation import?
a sidebar section	registerModule() in ui/admin/nav.ts	no — globbed
an admin page	admin/Pages/<Dir>/<Name>.tsx	no — globbed
a settings panel	registerSettingsTab()	no — globbed
a context provider	AdminLayout providers prop	no — passed in
a page title / actions	useSetPageHeader()	you import the hook

The three aliases @ui/* (shadcn), @lib/utils and @providers/theme come from the PROJECT, not from any plugin. Your plugin may use them; it must not ship them.

The one rule that generates all the others

A plugin never imports another plugin's UI in order to render inside a shared screen. If you feel the need to, the screen is missing a registry and you should add one.

2 Quick start: an admin page in one file

This is a complete, working admin page contributed by a plugin. Nothing else is needed for it to appear at /admin/rental/properties with the full shell — sidebar, header, breadcrumbs, toasts.

2.1 The page

```

// plugins/hkm-plugin-rental/ui/admin/Pages/Rental/Properties.tsx
import type { ReactNode } from "react";
import { usePage } from "@pageflow/react";
import { AdminLayout, useSetPageHeader, DataTable, type Column } from "@pageflow/admin";

interface Property {
  id: string;

```

```

reference: string;
addressLine: string;
status: "vacant" | "occupied" | "maintenance";
monthlyRentCents: number;
}

interface Props extends Record<string, unknown> {
  properties: Property[];
}

const money = (cents: number) =>
  new Intl.NumberFormat(undefined, { style: "currency", currency: "UGX" })
    .format(cents / 100);

export default function Properties() {
  const { props } = usePage<Props>();

  useSetPageHeader({
    title: "Properties",
    description: "Every unit in the portfolio",
  });

  const columns: Column<Property>[] = [
    { key: "reference", header: "Ref", sortable: true },
    { key: "addressLine", header: "Address", sortable: true },
    { key: "status", header: "Status", filterable: true,
      filterOptions: [
        { label: "Vacant", value: "vacant" },
        { label: "Occupied", value: "occupied" },
        { label: "Maintenance", value: "maintenance" },
      ] },
    { key: "monthlyRentCents", header: "Rent", className: "text-right",
      render: (p) => money(p.monthlyRentCents) },
  ];

  return (
    <div className="container py-6">
      <DataTable
        data={props.properties}
        columns={columns}
        searchKey={["reference", "addressLine"]}
        searchPlaceholder="Search by reference or ...address"
        emptyMessage="No properties yet."
      />
    </div>
  );
}

// The shell. Applied OUTSIDE the page by Pageflow, so it survives navigation.
Properties.layout = (page: ReactNode) => <AdminLayout>{page}</AdminLayout>;

```

2.2 The route

In the plugin's `module.json`. The route-level `requires` pulls in Pageflow for this route only — the module itself need not depend on it.

```

{
  "routePrefix": "/admin/rental",
  "routes": [
    { "method": "GET", "path": "/properties",
      "handler": "Plugins\\Rental\\Infrastructure\\Http\\Controllers\\RentalPageController@properties",
      "name": "rental.properties",

```

```
    "filters": ["auth"],
    "requires": ["http.pageflow"] }
  ]
}
```

2.3 The controller

```
<?php
declare(strict_types=1);

namespace Plugins\Rental\Infrastructure\Http\Controllers;

use AlfacodeTeam\PhpServicePlatform\Kernel\Http\Request;
use AlfacodeTeam\PhpServicePlatform\Kernel\Http\Response;
use Plugins\Pageflow\Http\PageflowResponder;
use Plugins\Rental\API\Contracts\PropertyServiceContract;
use Project\Http\Controllers\Concerns\InteractsWithGraphSeo;

final class RentalPageController
{
    use InteractsWithGraphSeo;

    public function __construct(
        private readonly PageflowResponder $pageflow,
        private readonly PropertyServiceContract $properties,
    ) {
    }

    /** GET /admin/rental/properties -> component "Rental/Properties" */
    public function properties(Request $request): Response
    {
        return $this->pageflow->render($request, 'Rental/Properties', 'admin', [
            'properties' => array_map(
                static fn ($p) => $p->toArray(),
                $this->properties->all(),
            ),
            // Admin screens are never indexed: correct title + noindex, no graph.
            'seoHead' => $this->seoPrivate('Properties', request: $request),
        ]);
    }
}
```

2.4 Wire it up

```
hkm plugins enable rental
hkm ui sync                # mirrors ui/ into frontend/plugins/rental + aliases
cd frontend && npm run dev -- --mode admin
```

The file path is the component name

admin/Pages/Rental/Properties.tsx becomes the key "Rental/Properties", which is exactly the string the controller renders. Rename the file and you must rename the string.

3 The layout

3.1 Attach it, do not wrap with it

This is the single most important convention in the whole foundation, and the most common thing ported code gets wrong.

```
// ✓ CORRECT — a persistent layout
export default function Properties() {
  return <div...></div>;
}
Properties.layout = (page: ReactNode) => <AdminLayout>{page}</AdminLayout>;

// ✗ WRONG — wrapping the page's own return
export default function Properties() {
  return (
    <AdminLayout>
      <div...></div>
    </AdminLayout>
  );
}
```

Pageflow applies `Component.layout` OUTSIDE the component it swaps (`ui/react/App.tsx`). Because the layout element is created outside the swap, React sees the same `AdminLayout` element type before and after a navigation and preserves its state. Wrapping inline puts the layout INSIDE the swapped subtree, so every navigation unmounts and remounts the entire sidebar.

What that costs, concretely, on every single click:

- the nav overflow calculation restarts from zero, so the sidebar visibly reflows
- any open submenu popover closes
- sidebar scroll position resets to the top
- the collapse/expand spring animation replays

Where this comes from

All 46 admin pages in HKM 0.3 wrapped inline, and none of them used the persistent-layout support the client already had. If you are porting a 0.3 page, this is the first thing to change.

3.2 AdminLayout options

```
<AdminLayout
  providers={[MapProvider, QueryProvider]} // outermost first
  headerActions={<GlobalSearch />} // right of the top bar
  sidebarHeader={<ConnectedOrgSwitcher />} // default: read-only <OrgSwitcher/>
  sidebarFooter={<StorageMeter />} // between nav and the user row
  onNotifications={() => setPanelOpen(true)} // omit → the bell is hidden
  onHelp={() => open("/docs")} // omit → the help button is hidden
  cLock={false} // hide the header clock
  defaultSidebarOpen={false}
  toastPosition="bottom-right"
/>
```

The sidebar slots are named `sidebarHeader` / `sidebarFooter` on purpose. Plain header and footer

would read as the PAGE header and footer, which belong to `useSetPageHeader` and `useSetPageFooter` — a different thing entirely.

providers: how a plugin injects its own dependency

If your pages need a context provider — a maps SDK, a query client, a websocket — do not add it to the shell. Pass it in, so applications that do not use your plugin never pay for it.

```
// plugins/hkm-plugin-maps/ui/index.ts exports MapProvider
import { MapProvider } from "@maps";

MapPage.layout = (page: ReactNode) => (
  <AdminLayout providers={[MapProvider]}>{page}</AdminLayout>
);
```

Why this prop exists

The 0.3 layout hard-wired a Google-Maps provider into the shell itself, so every admin application in every project paid a maps dependency in order to render a sidebar. If you ever find yourself editing the foundation to add a provider, use this prop instead.

3.3 AuthLayout

For screens that render before there is an identity — sign-in, registration, OAuth consent, email verification. Nav-free by design: there is no identity yet, so there is nothing to build a sidebar from.

```
import { AuthLayout } from "@pageflow/admin";

Login.layout = (page: ReactNode) => (
  <AuthLayout
    brand="Acme"
    links={[{ label: "Privacy", href: "/privacy" }]}
  >
    {page}
  </AuthLayout>
);
```

4 Shell data: the adminShell shared prop

AdminLayout takes no data props. Everything it renders — the user row, the tenant switcher, which nav sections are visible, where log-out posts to — comes from one reserved shared prop called `adminShell`. Your pages never thread it down.

4.1 Server side — share it once

```
<?php
// A project stage, or your plugin's Provider::boot() — anywhere that runs per
// request BEFORE the responder renders.

pageflow_share('adminShell', [
    'user' => [
        'id' => $identity->userId,
        'displayName' => $identity->fullName ?: $identity->username,
        'email' => $identity->email,
        'avatarUrl' => $identity->avatarUrl,
    ],
    'tenant' => $tenant, // ?['id','name','logoUrl','meta']
    'tenants' => $memberships, // switchable tenants; fewer than 2 hides the switcher
    'features' => $ctx->features ?? [], // proj.json features[]

    'logoutUrl' => '/auth/logout',
    'accountUrl' => '/account/profile',
    'settingsUrl' => '/admin/settings',
    'appName' => env('APP_NAME', 'Admin'),
    'notificationCount' => $unread,
]);
```

Identity already carries username, email, fullName and avatarUrl — the Auth plugin fills them at issuance — so the user block is a projection, not a database read. features comes from proj.json via DomainContext, so enabling a section is a deployment decision, not a code change.

4.2 Client side — read it

```
import { useAdminShell, useCurrentPath } from "@pageflow/admin";

function TenantBanner() {
    const { user, tenant, tenants, features, appName } = useAdminShell();
    const path = useCurrentPath(); // NOT window.location — see the trap below

    if (!tenant) return null;
    return <p>{user.displayName} · {tenant.name} · {path}</p>;
}
```

Every field comes back resolved, never undefined: user falls back to a guest, tenants to an empty array, the URLs to sensible defaults. A surface that shares nothing still renders a shell rather than crashing.

Never derive the path from window.location

window.location.pathname is read once during render and never updates, so anything computed from it goes stale the moment the user navigates client-side. Use useCurrentPath(), which reads usePage().url. In 0.3 this exact bug left the breadcrumb pointing at the previous page while the sidebar highlight moved on.

5 Contributing navigation

Create `ui/admin/nav.ts` in your plugin. The admin surface globs `/plugins/*/admin/nav.ts` and imports every match for its side effects, so nothing imports your file by name and the foundation never learns your plugin exists.

```
// plugins/hkm-plugin-rental/ui/admin/nav.ts
import { Building2, Wrench, Users } from "lucide-react";
import { registerModule, registerFeature, registerIcons } from "@pageflow/admin";

// Only needed if you use the string form of `icon` anywhere.
registerIcons({ Building2, Wrench, Users });

registerFeature({
  id: "rental",
  label: "Rental management",
  description: "Properties, tenants, leases and maintenance",
  badge: { label: "Beta", variant: "beta" }, // default badge for gated items
});

registerModule({
  id: "rental", // MUST be unique across every plugin
  sectionLabel: "Rental", // the sidebar heading
  order: 40, // <10 overview · -1089 domains · ≥90 system
  features: ["rental"], // whole section hidden unless the flag is on

  items: [
    {
      id: "properties",
      label: "Properties",
      icon: Building2,
      path: "/admin/rental/properties",
      parentPath: "/admin/rental/properties",
      children: [
        { id: "properties-all", label: "All properties", path: "/admin/rental/properties" },
        { id: "properties-create", label: "Add property", path: "/admin/rental/properties/create" },
        // Not rendered, but still highlights its parent when active:
        { id: "properties-edit", label: "Edit", path: "/admin/rental/properties/edit", hidden: true },
      ],
    },
    {
      id: "maintenance",
      label: "Maintenance",
      icon: Wrench,
      path: "/admin/rental/maintenance",
      badge: { label: "New", variant: "new" },
    },
    {
      id: "tenants",
      label: "Tenants",
      icon: "Users", // string form — resolved via registerIcons
      path: "/admin/rental/tenants",
      feature: "rental-tenants", // gate ONE item independently
    },
  ],
});
```

5.1 How visibility is decided

The enabled feature set comes from the server and travels as `adminShell.features`:

```
proj.json  "features": [ ... ]
  |
  ▼
DomainContext->features      (project layer, resolved from the host)
  |
  ▼
pageflow_share('adminShell', ['features' => $ctx->features])
  |
  ▼
selectNavSections(features)   (the sidebar, during render)
```

```
// projects/<name>/proj.json
{
  "features": [
    "rental",
    "rental-tenants",
    { "id": "maintenance", "badge": { "label": "Preview", "variant": "beta" } }
  ]
}
```

The object form re-badges by id — and it works on an ITEM id as well as a feature id, which is how a deployment marks one page as Beta without a code change.

5.2 Badge resolution order

Priority	Source
1	the item's own badge field
2	a runtime badge keyed by the ITEM id
3	a runtime badge keyed by the item's feature
4	the registered feature's default badge

Two things that will bite you

- **Module ids are global.** Two plugins registering `id: "settings"` silently replace one another — one section just never appears. Replacement is deliberate (it is how a project overrides a plugin's section), so it cannot be an error. It warns in dev; namespace your id with the plugin slug.
- **A flag that matches nothing does nothing.** Enabling `"rentals"` when the feature is `"rental"` produces no error and no section. The registry logs the unmatched ids once — read the console.

5.3 Nav-only, no page yet

Do not register nav for routes that do not exist. In 0.3 five modules contributed roughly 115 items whose routes were never registered; every one of them 404'd, and the project's `proj.json` carried a comment explaining why they had to stay disabled. Register the section when the route lands.

6 Contributing a settings tab

A settings screen is the one place where several plugins genuinely want to render into one page: company details belong to Settings, SMTP to Mail, domains to Tenancy, clients to OAuth2, sessions to Auth. Writing that as imports would make the Settings plugin depend on five others.

So it is the same seam as navigation, with its own registry.

```
// plugins/hkm-plugin-mail/ui/admin/nav.ts
import { registerSettingsTab, type SettingsTabProps } from "@pageflow/admin";
import { Input } from "@ui/input";
import { Label } from "@ui/label";

function EmailSettingsPanel({ value, onChange, saving }: SettingsTabProps) {
  return (
    <div className="space-y-4">
      <div className="space-y-1.5">
        <Label htmlFor="smtp-host">SMTP host</Label>
        <Input
          id="smtp-host"
          disabled={saving}
          value={String(value.smtpHost ?? "")}
          onChange={(e) => onChange({ smtpHost: e.target.value })}
        />
      </div>
    </div>
  );
}

registerSettingsTab({
  id: "mail.smtp",          // namespace with your slug – ids are global
  label: "Email",
  group: "Communication",
  order: 30,
  feature: "mail",          // hidden unless the flag is on
  component: EmailSettingsPanel,
});
```

The settings PAGE reads them back and owns dirty-tracking and saving; your panel only reports changes through `onChange`.

```
// whichever page renders the settings screen
import { selectSettingsTabs, useAdminShell } from "@pageflow/admin";
import { Tabs, TabsContent, TabsList, TabsTrigger } from "@ui/tabs";

const { features } = useAdminShell();
const tabs = selectSettingsTabs(features); // ordered, feature-gated

<Tabs defaultValue={tabs[0]?.id}>
  <TabsList>
    {tabs.map((t) => <TabsTrigger key={t.id} value={t.id}>{t.label}</TabsTrigger>)}
  </TabsList>
  {tabs.map(({ id, component: Panel }) => (
    <TabsContent key={id} value={id}>
      <Panel value={settings} onChange={patch} saving={saving} />
    </TabsContent>
  ))}
</Tabs>
```

7 Page header and footer

The page declares what it wants; the layout decides where it renders. Neither imports the other. Copy this shape if you ever need a new shell slot — it is the reason a plugin page can put a Save button in the application chrome without the chrome knowing that plugin exists.

```
import { useSetPageHeader, useSetPageFooter } from "@pageflow/admin";
import { Button } from "@ui/button";
import { Plus, Save } from "lucide-react";

useSetPageHeader({
  title: "Properties",
  description: "Every unit in the portfolio",
  actions: [
    { label: "Import", onClick: openImport, variant: "outline" },
    { label: "Add property", onClick: openCreate, icon: <Plus className="h-4 w-4" /> },
  ],
}, [openImport, openCreate]); // ← deps matter, see below

// A sticky bar at the bottom – bulk actions, unsaved-changes prompts.
useSetPageFooter({
  content: (
    <div className="flex items-center justify-between">
      <span>{selected.length} selected</span>
      <Button onClick={save}><Save className="mr-2 h-4 w-4" /> Save</Button>
    </div>
  ),
}, [selected.length, save]);
```

The deps array is not optional in practice

`useSetPageHeader` takes a dependency list that defaults to `[]`. With the default, the header is set once on mount and never updates — so a title that depends on loaded data ("Invoice #4821") will stay on its placeholder forever. Pass the values the header reads.

Both hooks clear on unmount, so navigating away never leaves the previous page's title in the bar. `actions` accepts either the array form above or arbitrary JSX when you need something a button cannot express.

8 Lists, tables and mutations

8.1 Which one to reach for

Component	Use when
DataTable	you want sorting, column filters, a row-action menu and delete confirmation out of the box
ResourceListShell + SimpleTable	you want the full page chrome (stats, chips, empty state) and full control of the row markup
useResourceList	client-side search / status filter / pagination state for either of the above
TableQuery	the dataset is too large to hold, and the server does the filtering

8.2 A complete CRUD page

This is the pattern most plugin list screens should follow. It uses the shell for chrome, `useResourceList` for state, and `resourceRequest` for mutations.

```
// plugins/hkm-plugin-rental/ui/admin/Pages/Rental/Properties.tsx
import { useState, type ReactNode } from "react";
import { usePage } from "@pageflow/react";
import {
  AdminLayout, useSetPageHeader,
  ResourceListShell, SimpleTable, useResourceList, resourceRequest,
} from "@pageflow/admin";
import { Building2, CheckCircle2, Wrench } from "lucide-react";
import { Button } from "@ui/button";

interface Property {
  id: string; reference: string; addressLine: string;
  status: "vacant" | "occupied" | "maintenance";
}

interface Props extends Record<string, unknown> { properties: Property[] }

const STATUSES = [
  { value: "all", label: "All" },
  { value: "vacant", label: "Vacant", icon: Building2 },
  { value: "occupied", label: "Occupied", icon: CheckCircle2 },
  { value: "maintenance", label: "Maintenance", icon: Wrench },
];

export default function Properties() {
  const { props } = usePage<Props>();
  const [busy, setBusy] = useState(false);

  // Adopts server-sent rows when props change — see the trap below.
  const list = useResourceList<Property>({
    initial: props.properties,
    getId: (p) => p.id,
    searchFields: (p) => [p.reference, p.addressLine],
    getStatus: (p) => p.status,
    pageSize: 20,
  });

  useSetPageHeader({ title: "Properties" }, []);

  const remove = (property: Property) => {
    setBusy(true);
    resourceRequest({
      url: `/ajax/rental/properties/${property.id}`,
    });
  };
}
```

```

    method: "delete",
    failureMessage: "Could not delete that property",
    successMessage: "Property deleted",
    onOk: () => list.removeItem(property.id),
    onSettled: () => setBusy(false),
  });
};

return (
  <ResourceListShell
    stats={[
      { label: "Total", value: list.counts.all ?? 0, icon: Building2 },
      { label: "Occupied", value: list.counts.occupied ?? 0, icon: CheckCircle2,
        color: "bg-emerald-500/10 text-emerald-600" },
    ]}
    search={list.search}
    onSearchChange={list.setSearch}
    searchPlaceholder="Search by reference or ...address"
    createLabel="Add property"
    onCreate={() => router.visit("/admin/rental/properties/create")}
    filters={{
      options: STATUSES, value: list.status,
      onChange: list.setStatus, counts: list.counts,
    }}
    empty={{
      icon: Building2,
      title: "No properties yet",
      description: "Add your first unit to start tracking occupancy.",
      actionLabel: "Add property",
      onAction: () => router.visit("/admin/rental/properties/create"),
    }}
    isEmpty={list.filtered.length === 0}
    page={list.page}
    totalPages={list.totalPages}
    totalResults={list.filtered.length}
    onPageChange={list.setPage}
  >
    <SimpleTable
      columns={[
        { key: "ref", label: "Reference" },
        { key: "addr", label: "Address", hideOnMobile: true },
        { key: "status", label: "Status" },
        { key: "act", label: "", align: "right" },
      ]}
    >
      {list.paginated.map((p) => (
        <tr key={p.id}>
          <td className="px-4 py-3 font-medium">{p.reference}</td>
          <td className="hidden px-4 py-3 sm:table-cell">{p.addressLine}</td>
          <td className="px-4 py-3">{p.status}</td>
          <td className="px-4 py-3 text-right">
            <Button variant="ghost" size="sm" disabled={busy}
              onClick={() => remove(p)}>Delete</Button>
          </td>
        </tr>
      ))}
    </SimpleTable>
  </ResourceListShell>
);
}

Properties.layout = (page: ReactNode) => <AdminLayout>{page}</AdminLayout>;

```

preserveState: the trap behind two separate bugs

`router.post`, `router.put` and `router.patch` all default to `preserveState: true`, and `resourceRequest` sets it explicitly. That means the page component is **not remounted** after a mutation.

Consequences you must plan for:

- Any `useState(props.x)` you write by hand keeps the value captured on first mount. `useResourceList` handles this for you via `syncOnPropChange` (default `true`); hand-rolled state does not.
- A `router.reload({ only: ['rows'] })` will appear to do nothing at all if your state was seeded once and never re-synced.

8.3 Server-side tables

When the dataset is too large to send, switch `DataTable` into server mode. Every parameter change — debounced search, sort, column filter, page — arrives in one callback.

```
const [rows, setRows] = useState(props.rows);
const [total, setTotal] = useState(props.total);
const [loading, setLoading] = useState(false);

<DataTable
  serverSide
  data={rows}
  totalRecords={total}
  loading={loading}
  columns={columns}
  onFetchData={({ start, length, search, filters, sortKey, sortDirection }) => {
    setLoading(true);
    router.visit("/admin/rental/properties", {
      method: "get",
      data: { page: start + 1, per_page: length, q: search,
        sort: sortKey, dir: sortDirection,
        ... Object.fromEntries(filters.map((f) => [f.key, f.value])) },
      only: ["rows", "total"],
      preserveState: true,
      preserveScroll: true,
      onSuccess: (page) => { setRows(page.props.rows); setTotal(page.props.total); },
      onFinish: () => setLoading(false),
    });
  }}
/>
```

`TableQuery` is the immutable helper for holding that query state. Every mutator returns a new instance, so it is safe in React state — and `toParams()` produces the flat object above.

```
import { TableQuery } from "@pageflow/admin";

const [query, setQuery] = useState(() => TableQuery.from({ length: 25 }));

setQuery((q) => q.setFilter("status", "vacant")); // resets to page 1
setQuery((q) => q.toggleSort("reference")); // unsorted → asc → desc → off
router.visit(url, { data: query.toParams() });
```

8.4 Mutations: resourceRequest

```
resourceRequest<Property>({
```

```

url: "/ajax/rental/properties",
method: "post",
data: { reference, addressLine },
resultKey: "property",          // prop holding the created row
only: ["stats"],                // anything else you want refreshed
successMessage: "Property created",
failureMessage: "Could not create that property",
onOk:      (created) => created && list.addItem(created),
onInvalid: (fields) => setFieldErrors(fields),    // 422, field-level
onSettled: ()      => setBusy(false),
});

```

It handles both shapes a Sentinel endpoint can answer with — a populated `errors` bag, or the `{ error: { code, message, fields } }` envelope arriving as a prop — so no call site has to know which.

error is a reserved prop name

`resourceRequest` always requests a prop called `error` and reads it as a failure envelope. If your page uses `error` as an ordinary prop, rename it.

9 Form controls and hooks

9.1 Controls

```

import { FieldHelp, TagInput, SuggestionSelect } from "@pageflow/admin";

// A "?" button beside a label, sized for a wide age range.
<FieldHelp
  title="Property reference"
  rules={['Must be unique', 'Letters, numbers and dashes only']}
  example="BLK-A-12"
>
  The short code your team uses to refer to this unit.
</FieldHelp>

// Tag entry: Enter adds, Backspace on empty removes the last chip.
<TagInput
  label="Amenities"
  items={amenities}
  onChange={setAmenities}
  tone="primary"
  help={{ title: "Amenities", rules: ["One per chip", "Keep them short"] }}
/>

// A dropdown of suggestions over a FREE-TEXT column, with an escape hatch.
<SuggestionSelect
  value={heatingType}
  onChange={setHeatingType}
  options={[{ label: "Central", value: "central" },
    { label: "Solar", value: "solar" }]}
/>

```

`SuggestionSelect` exists for a specific failure. A closed dropdown over a free-text column silently rewrites history: open an old record whose value is not on the list, the control shows blank, and saving writes that blank over a real answer. This one starts in custom mode with the original text intact.

9.2 Hooks

Hook	What it gives you
useAdminShell()	resolved shell data — user, tenant(s), features, URLs
useCurrentPath()	the current pathname from usePage(), never stale
useIsMobile() / useMediaQuery()	breakpoint state, correct on first paint
useDebounceAutosave()	debounced save with status + a flush() for cmd-S
useApi()	data that is NOT in the page payload (search-as-you-type)
usePageflowErrors()	already mounted by the layouts — see below

```
import { useDebounceAutosave } from "@pageflow/admin";

const { status, lastSavedAt, flush } = useDebounceAutosave({
  value: draft,
  save: async (v) => { await api.saveDraft(v); },
  delay: 1200,
  enabled: !loading,
  isEqual: (a, b) => JSON.stringify(a) === JSON.stringify(b),
});

// status: "idle" | "pending" | "saving" | "saved" | "error"
```

Do not toast validation errors

A 422 travels two paths at once: the client fires the global `error` event **and** calls the visit's `onError`, which is what `useForm` and `<Form>` use to put a message under each field.

So `usePageflowErrors` — already mounted for you by the layouts — stays silent on validation by default and only toasts transport failures, which have no field to attach to. Adding your own toast on top would report every failed form submission in the application twice. If a surface submits through bare `router.post` calls where nothing renders the errors, pass `validationToasts: true`.

10 Shipping both faces

A plugin can serve the admin surface and the public site from one `ui/`. Declare the map in `ui.json`; each surface globs the pages for its own face.

```
{
  "alias": "@rental",
  "entry": "index.tsx",
  "framework": "react",
  "surfaces": {
    "admin": "admin/Pages",
    "site": "site/Pages"
  },
  "dependencies": {
    "@dnd-kit/core": "^6.0.0"
  }
}
```

```
plugins/hkm-plugin-rental/ui/└─
admin/Pages/Rental/Properties.tsx → "Rental/Properties" (AdminLayout)└─
site/Pages/Rental/Listing.tsx → "Rental/Listing" (your own layout)└─
components/PropertyCard.tsx shared by both faces└─
index.ts export { PropertyCard } → "@rental"
```

Public pages must NOT use `AdminLayout` — it would pull the whole admin shell into a marketing bundle. Give them their own layout, or none.

Declare your own dependencies in `ui.json`. A vendor SDK belongs to the plugin that needs it, never to the foundation — that is the whole reason the foundation ships only `lucide-react`, `framer-motion` and `sonner`.

10.1 Overriding a plugin page from a project

Surfaces spread the project's own pages FIRST, so declaring the same component name in the project wins. Nothing in the plugin changes.

```
frontend/src/surfaces/admin/Pages/Rental/Properties.tsx ← project's version wins
frontend/plugins/rental/admin/Pages/Rental/Properties.tsx ← plugin's, now shadowed
```

To remove a plugin route entirely rather than replace its page, use the project's disable policy — no fork required:

```
// proj.json
{
  "routePolicy": {
    "disable": [
      "GET /admin/rental/maintenance", // one route
      "rental.management"             // or a whole module's routes, by domain
    ]
  }
}
```

A disable spec that matches nothing fails the boot

That is deliberate — it is the anti-typo guard. If the build fails here, check the spelling against the compiled route manifest.

11 The trap list

Every item below is a defect that has actually been found in this codebase, not a hypothetical. They are the ones a type-checker cannot see.

What looks right	What actually happens
wrapping the page in <code><AdminLayout></code> <code>useState(props.rows)</code>	the sidebar remounts on every navigation stale after any mutation — <code>preserveState</code> is true by default
toasting validation errors <code>router.visit(target)</code> after sign-in	double-reports; <code>useForm</code> already renders them inline carries a stale CSRF token and stale shell props; use a full load
<code>window.location.pathname</code>	read once at render; goes stale after client-side navigation
<code><a href></code> to any endpoint assuming <code>item.\$id</code>	405 if the route is POST-only — read <code>module.json</code> first a 0.3 storage artefact; identity defaults to <code>item.id</code> , or pass <code>getId</code>
importing <code>AppErrorBoundary</code> from <code>@pageflow/admin</code> <code><Head title="..."></code>	pulls the whole shell; it lives in <code>@pageflow/react</code> titles are server-driven via the <code>seoHead</code> prop
a trait method you have seen elsewhere	traits compose — find the trait that <code>DECLARES</code> it

Why this list exists

A review of roughly 5,600 lines of freshly written code for this foundation found **fifteen defects**. `tsc --noEmit` was clean throughout and caught **none** of them; `php -l` caught none. Not one was a typo. Each was a coherent, plausible belief that happened to be false — a method on the wrong trait, a link to a POST-only route, a framework default that was the opposite of what its API implied. Before you report a page as working, open the definition of what it calls: the trait, the signature, the route's METHOD, the client's actual default. And say plainly what you did not verify — *type-checks clean* is not evidence that anything works.

12 Checklist and cheat sheet

12.1 Adding an admin page to a plugin

1. Create `ui/admin/Pages/<Dir>/<Name>.tsx` — the path IS the component name.
2. Attach the shell: `Page.layout = (p) => <AdminLayout>{p}</AdminLayout>`.
3. Declare the title with `useSetPageHeader({ ... }, [deps])`. **No** `<Head title>`.
4. Add the route to `module.json` with `requires: ["http.pageflow"]` and a `filters: ["auth"]` where appropriate.
5. Render it from a controller: `$this->pageflow->render($request, '<Dir>/<Name>', 'admin', [ ... ])` with a `seoHead` from `seoPrivate()`.
6. Register nav in `ui/admin/nav.ts` — but only once the route exists.
7. Run `hkm ui sync`, then `npm run dev -- --mode admin`.

12.2 Cheat sheet

Shell	Page.layout = (p) => <AdminLayout>{p}</AdminLayout>
Shell data	const { user, tenant, features } = useAdminShell()
Current path	const path = useCurrentPath() // never window.location
Title + actions	useSetPageHeader({ title, actions }, [deps])
Sticky footer	useSetPageFooter({ content }, [deps])
Sidebar section	registerModule({ ... }) in ui/admin/nav.ts
Settings panel	registerSettingsTab({ ... })
Own provider	<AdminLayout providers={[MyProvider]}>
List chrome	<ResourceListShell ...><SimpleTable ...>
List state	const list = useResourceList({ initial, getId, searchFields })
Batteries table	<DataTable data columns getId serverSide onFetchData />
Mutation	resourceRequest({ url, method, onOk, onInvalid })
Error boundary	import { AppErrorBoundary } from "@pageflow/react"
Sync after enabling	hkm plugins enable <name> && hkm ui sync
Dev	npm run dev -- --mode admin
Build all surfaces	npm run build:all

12.3 Further reading

Document	Covers
plugins/hkm-plugin-pageflow/ui/admin/README.md	every export, in reference form
tools/src/templates/frontend/docs/HOW_IT_WORKS.md	surfaces, federation, adding a page
plugins/hkm-plugin-user/ui/README.md	a worked two-face plugin
plugins/hkm-plugin-auth/ui/README.md	the default sign-in page and how to replace it
docs/migration/ADMIN-UI-DISTRIBUTION.md	the 0.3 map, the layout in detail, the defect log
CLAUDE.md	the platform contract, including the verification rule